Student Name: ARVIND VIKRAM M S

ROLL NO: 7376222CB104

Seat No:179

Project ID: 19

Project title: (Transport)BIT Bus Management

STACK: SPRING BOOT

Frontend	React
Backend	Spring boot
Database	MySQL
Security	Spring Security

PROBLEM STATEMENT:

The current system for managing bus seating for Dayscholar students is often manual, inefficient, and lacks organization. This leads to several challenges:

- **Inefficient Seat Allocation:** Traditional methods may involve manual selection or a first-come-first-served approach, leading to potential conflicts and a lack of organization.

- **Inaccurate Attendance Tracking:** Attendance might be recorded manually, which can be time-consuming and prone to errors.

- **Limited Communication:** Students may not receive timely or clear information regarding their allocated seat or changes to bus routes or schedules.

- **Inflexible System:** The current process might be inflexible when dealing with temporary situations like route changes or merging buses for specific needs

SOLUTION:

The Bus Seating Management App offers a digital solution: automated seat booking, attendance tracking, clear communication via email notifications, and flexible bus management for temporary route merging. This app aims to streamline student transportation for everyone involved.

PROJECT FLOW:

Scope:

The application caters to three user groups:

- **Students:** Book buses for the semester, view allocated seats, and receive seat notifications.
- **Faculty:** Select assigned bus, view student lists, and mark student attendance.
- **Administrators:** Create and manage buses, assign faculty, manage seat allocations, initiate bus merging based on criteria, and trigger seat allocation for merged buses with email notifications.

Purpose:

The Bus Seating Management App aims to:

- **Automate seat booking and allocation:** Ensure fairness and organization through an algorithm-based approach.
- **Enhance faculty attendance tracking:** Facilitate faster and more accurate attendance recording.
- **Improve communication:** Provide timely information to students regarding seat allocation and route changes.
- **Offer administrative control:** Empower administrators to manage buses, faculty assignments, and student seat allocations, including temporary route merging capabilities.

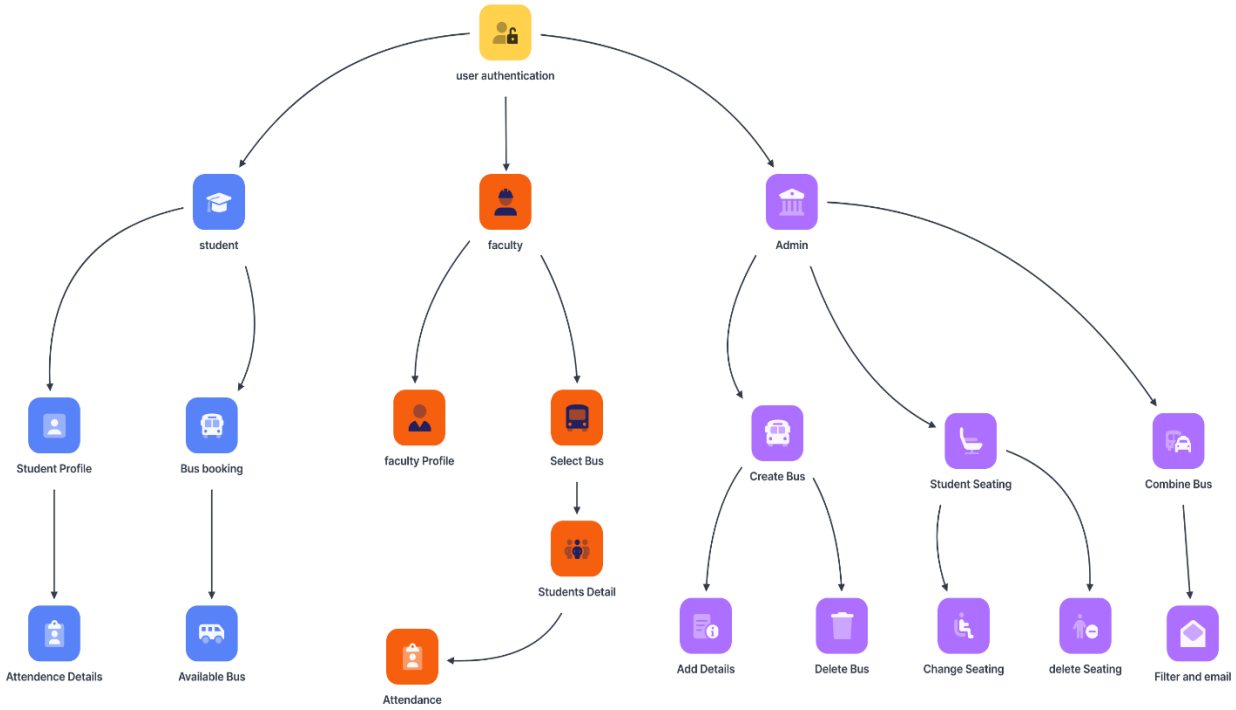
Target Audience:

Students: This application is primarily targeted towards dayscholar students who utilize the institution's bus transportation system.

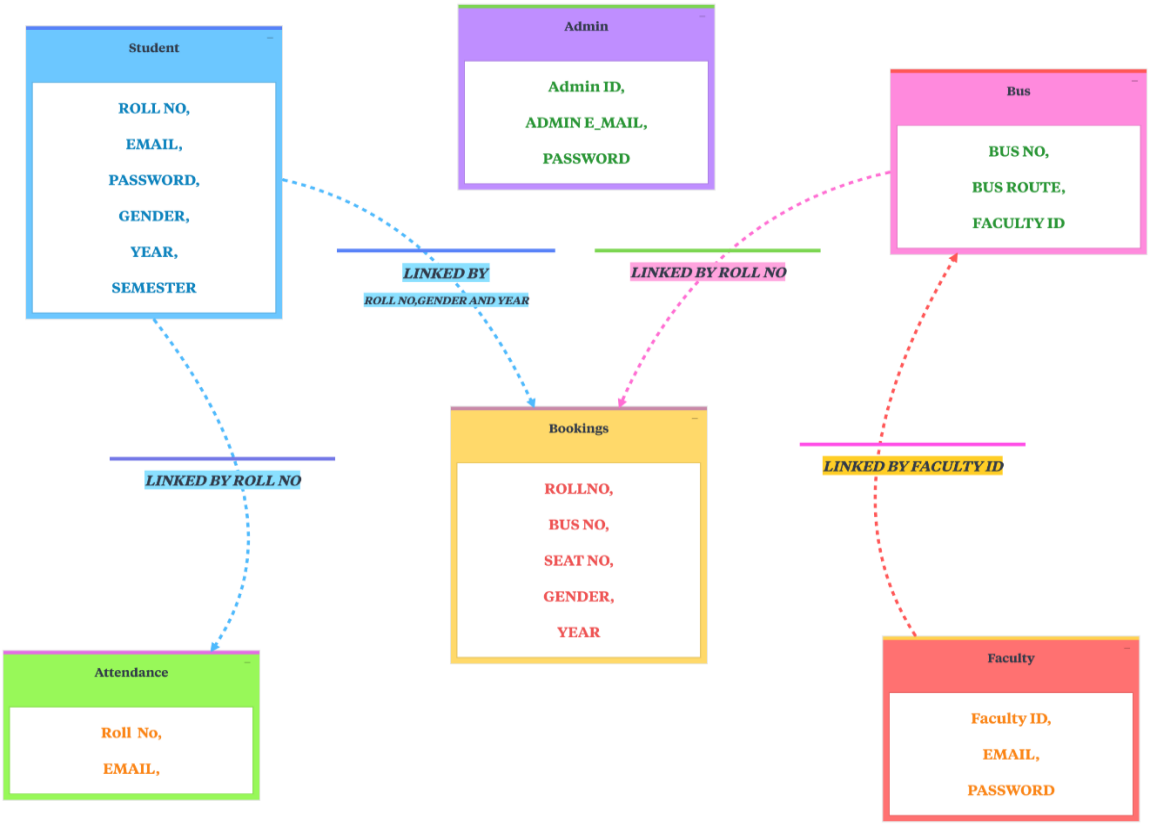
Faculty: The app caters to faculty members assigned to oversee specific buses.

Administrators: The app equips administrators with comprehensive management capabilities.

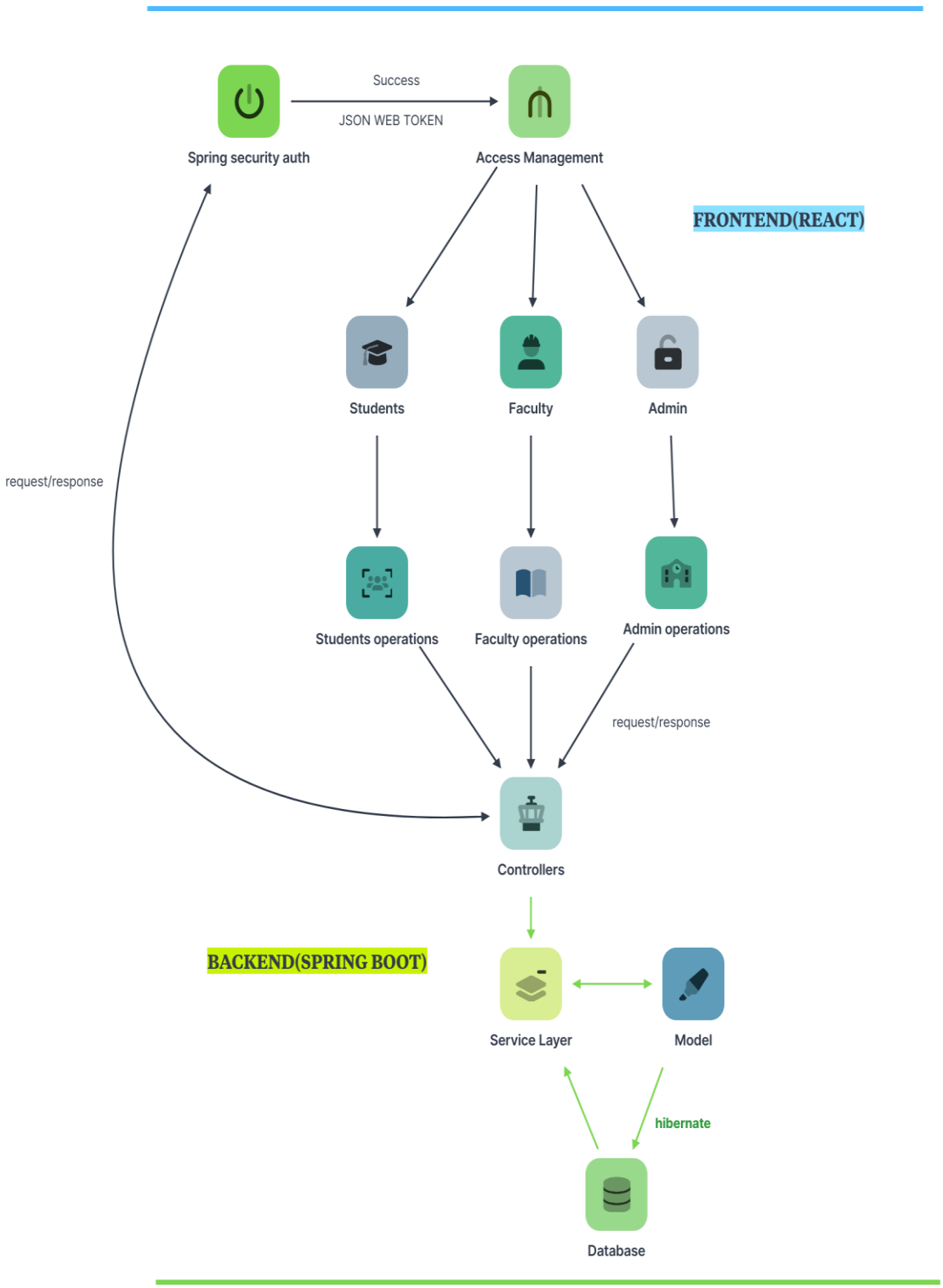
SITE MAP:



Database Design:



WORKFLOW:


Success
JSON WEB TOKEN
Spring security auth
Access Management
FRONTEND(REACT)
Students
Faculty
Admin
request/response
Students operations
Faculty operations
Admin operations
request/response
Controllers
BACKEND(SPRING BOOT)
Service Layer
Model
hibernate
Database

FUNCTIONAL REQUIREMENTS:

Student:

- **Login:** Students can log in using their email address.
- **Bus Booking:** Students can book their preferred bus for the semester. The system should enforce limitations to prevent duplicate bookings per student and semester.
- **View Allocated Seat:** Once a seat is allocated, students can access their profile within the app to view their assigned seat number.
- **Seat Notification:** Students will receive email notifications confirming their allocated seat number.

Faculty:

- **Select Assigned Bus:** Faculty members can select the specific bus they are assigned to oversee.
- **View Student List:** Faculty can view a list of students registered for their assigned bus. This list should display relevant student information for attendance purposes.

Admin:

- **Create/Manage Buses:** Admins can create new buses, defining a limit to ensure efficient management. They can also manage existing buses, potentially editing routes, timings, or capacity.
- **Assign Faculty:** Admins have the authority to assign faculty members to specific buses.
- **Manage Seat Allocations:** Admins can view and manage student seat allocations. This includes functionalities to change or delete existing allocations if necessary.
- **Bus Merging:** Admins can initiate the temporary merging of bus routes based on defined criteria (e.g., student year).
    - **Define Merging Criteria:** Admins can select two buses to be merged and define student criteria (like year) to filter the student list for the merged group.

- **View Merged Student List:** A separate temporary table within the database will store the list of students based on the merging criteria. This list will be used for seat allocation during the merged route period without affecting the original seat allocation data (This ensures data integrity and avoids affecting regular student seat assignments).

- **Trigger Seat Allocation (Merged Buses):** Admins can trigger seat allocation for the merged student list. The system will allocate seats based on the pre-defined algorithm (considering gender preference and seat position logic). Students will receive email notifications with their allocated seat number for the merged route.

NON-FUNCTIONAL REQUIREMENTS:

Performance:

- **App Responsiveness:** The application should be responsive and provide a smooth user experience across all functionalities. Page load times and response times for user actions should be optimized for a variety of devices (desktops, laptops, mobiles).

- **Data Processing Speed:** Data processing tasks like seat allocation and user data retrieval should be efficient. Consider implementing caching mechanisms for frequently accessed data to improve performance.

Security:

- **User Authentication:** Spring Security is implemented to ensure secure user authentication using student email addresses. JWT (JSON Web Tokens) is used for secure session management.

- **Data Encryption:** Implementing data encryption at rest for sensitive student data (like email addresses). This adds an extra layer of security in case of a data breach.

- **Privacy Considerations:** The application handles the student data responsibly. Data collection is limited to what's necessary for the app's functionality (e.g., email address, name, year for merging).

Usability:

- The user interface (potentially built with React) should be intuitive and user-friendly for all user roles (students, faculty, admins).

- Clear navigation, informative messages, and well-defined functionalities are crucial for a positive user experience.

Scalability:

- The application is designed to accommodate a growing number of users (students, faculty) and buses. The chosen technologies (Spring Boot for the backend and a relational database like MySQL) offer inherent scalability to handle increasing data volumes and user traffic.

- **Advantages of Spring Boot and Hibernate:**
    - Spring Boot provides a lightweight and efficient framework for building web applications.
    - Hibernate simplifies database interactions and object-relational mapping, improving data persistence and scalability.
    - Spring MVC offers a clear separation of concerns between the user interface and the backend logic, promoting maintainability and scalability.

DEPENDENCIES:

Front-End:

- **React (or similar):** A JavaScript library for building user interfaces. React allows for the creation of reusable components and provides a declarative approach to UI development.

Back-End:

- **Spring Boot:** A popular framework for building web applications in Java.

- **Spring Security :** A security framework for Spring Boot applications that offers user authentication, authorization, and access control functionalities.

- **Hibernate:** An object-relational mapper (ORM) that simplifies communication between the application and the database. Hibernate maps Java objects to database tables and handles data persistence.

Database:

- **MySQL:** A popular relational database management systems (RDBMS) that can store and manage the application's data.